

Healing Pipeline — Premature-Exit Audit

Scope: every place from `ExecutionEngine` → `Validation` where healing can terminate, short-circuit, or refuse a locator swap (`return` / `continue` / `break` / `report_only` / `skip` / `healable=false`).

Method: static read of the actual code paths. No code was changed.

Legend for “Should it be a hard stop or an advisor?”

- **HARD STOP** = correct to terminate; locator healing genuinely cannot help.
 - **ADVISOR** = should not terminate the pipeline; it should emit a signal/score and let later layers still try a grounded locator swap.
 - **GREY** = depends on evidence quality; currently a stop, arguably an advisor.
-

Stage 0 — ExecutionEngine (`src/core/execution-engine.ts`)

#	Location	Exit	Evidence that triggers it	Could locator healing still succeed?	Verdict
E1	<code>installDependencies</code> throws (<code>no package.json</code> , <code>npm install</code> failed x2 , <code>node_modules</code> missing)	throw → job fails	filesystem / npm state	No — nothing can run	HARD STOP
E2	<code>run / run-Async</code> timeout → <code>exitCode 124</code> , often no results file	killed at <code>HEALING_RERUN_TIMEOUT_MS</code> (120s)	wall-clock only	Maybe — a slow page can look like a timeout; the original break may still be a locator	GREY — currently indistinguishable from a real hang
E3	<code>exitCode 127</code> (command not found)	logged, returns empty-ish result	launcher/PATH	No	HARD STOP (but should be labeled <code>environment</code> , not flow into <code>framework / unknown</code>)
E4	No <code>test-results.json</code> produced and stdout isn't JSON	returns result with empty report	crash before reporter init (xvfb/xauth, OOM, bad flag)	N/A for this run , but it must not be read as the test's verdict	GREY — see Cross-cutting #1

Key risk: E2 + E4 both yield “a run with no/again-failing result” that downstream cannot distinguish from “the locator is still broken.” This is the exact ambiguity that turned execution 118 into a `framework /report-only`.

Stage 1 — ArtifactCollector (`src/core/artifact-collector.ts`)

#	Location	Exit	Evidence	Could locator healing still succeed?	Verdict
A1	<code>collect()</code> throws if <code>test-res-ults.json</code> missing (L68)	throw	file missing	No	HARD STOP (correct — but caller should treat as “no verdict”, not “framework”)
A2	<code>walkSuites</code> skips passed / skipped results (L96)	continue	result status	n/a	CORRECT
A3	A failure with no parseable locator → <code>failed_locator: null</code> (L211)	not an exit itself, but starves every grounded advisor	Playwright didn’t echo a locator AND no Page-Object resolution	Yes — App Profile / DOM could still ground a selector from the line or URL	ADVISOR — null locator should downgrade confidence, not silently disable healing
A4	<code>findActualSourceLocation</code> falls back to spec file when no PO frame found (L343)	wrong <code>file_path</code>	stack has no PO frame	Heal may target the wrong file → validation “locator not in file”	GREY — mis-resolution masquerades as “unhealable”
A5	Top-level load errors live in <code>errors[]</code> , not walked by <code>collect()</code> (L456 helper exists but <code>collect</code> ignores it)	0 artifacts collected	spec throws at import / global-setup fails	No (test never ran)	HARD STOP , but currently silent — surfaces as “0 failures”

Key risk: A3 is the quiet one — a null `failed_locator` doesn’t stop the pipeline, it defangs it: rule/pattern/validation/DOM all have nothing to anchor

on, so the only survivor is the AI (or nothing), and the diagnosis slides toward
`unknown` → `report_only`.

Stage 2 — FailureAnalyzer + FailureClassifier (`failure-analyzer.ts` , `failure-classifier.ts`)

#	Location	Exit / verdict	Evidence	Could locator healing still succeed?	Verdict
C1	<code>detectFailureType</code> → <code>unknown</code> (analyzer L130)	not an exit; sets category <code>unknown</code>	error text matched none of locator/timeout/assert/nav patterns	Often yes — “target closed” after a real locator wait still has <code>waiting for locator(...)</code> earlier in the joined message	GREY — the regex is first-match-wins on a joined multi-error string
C2	<code>looksLikeFrameworkFailure</code> → <code>category=framework</code> , confidence 0.8 (classifier L230)	<code>report_only</code> downstream	<code>target</code> closed/ crashed, <code>protocol error</code> , <code>browser-Type.launch</code> , <code>executable</code> doesn't exist	Sometimes — a crash can happen after a locator timeout; the framework signal then masks a real locator break	GREY → ADVISOR — framework should be a signal, not a terminal category, unless it's the ONLY evidence
C3	<code>looksLikeApiFailure</code> / <code>looksLikeEnvironmentFailure</code> → <code>api</code> / <code>environment</code> (L233-239)	<code>report_only</code>	<code>request/status/env-var</code> regex	Rarely	GREY — same masking risk as C2
C4	<code>healableByLocatorSwap</code> = <code>category==='locator' && !!locator</code> (L286)	gates all swap healing	category + locator presence	This is THE gate — any mis-categorization here kills healing	ADVISOR territory — see Cross-cutting #2
C5	<code>refineDiagnosis</code> <code>isWithEvidence</code> : !	evidence ignored if <code>loc-</code>	<code>EvidenceCollector</code> produced no <code>locatorState</code>	Yes — exactly execution-118's case: <code>locator-</code>	GREY — no evidence ⇒ classifier should defer,

#	Location	Exit / verdict	Evidence	Could locator healing still succeed?	Verdict
	ls.exists only fires when ls.source != 'unknown' (L372)	ator- State=null	(no DOM/live probe)	orState: null ⇒ evidence stage is a no-op ⇒ parser verdict stands	not let a weak parser verdict become final
C6	strategyFor- Category : everything except locator/timing → report_only (L327)	report_only	category	n/a (consequence of C1-C4)	Consequence

Key risk (matches your hypothesis, but inverts the blame): the evidence refinement (C5) is **disabled whenever locatorState is null** — and locatorState is null precisely when there was no DOM snapshot / live probe (framework crash, fast failure). So a weak parser verdict (unknown → framework) is never corrected by evidence. The shape isn't corrupted; the evidence stage is skipped, leaving the parser's first guess as the final word.

Stage 3 — Strategy Router (`healing-strategy-router.ts`)

#	Location	Exit	Evidence	Could locator healing still succeed?	Verdict
R1	Guard 1: confidence < 0.5 && category! == 'locator' → report_only (L78)	report_only	low confidence	Yes — low confidence is a reason to gather more, not to refuse	ADVISOR
R2	case 'locator' but ! healableBy-LocatorSwap \\ \\ !locator → report_only (L103)	report_only	no concrete locator	Maybe (App Profile could ground from URL)	GREY
R3	assertion → report_only (L132)	report_only	element found, value mismatch	No (real product defect)	HARD STOP ✓
R4	navigation → report_only (L137)	report_only	net::ERR, navigation	No	HARD STOP ✓
R5	api → report_only (L142)	report_only	request/status	No	HARD STOP ✓
R6	environment → report_only (L147)	report_only	env/cred/permission	No	HARD STOP ✓
R7	framework → report_only (L152)	report_only	C2 signal	Sometimes (C2 masking)	GREY → ADVISOR
R8	unknown /default → report_only (L157)	report_only	nothing matched	Often — “unclassified” ≠ “unhealable”	ADVISOR — biggest false-negative source

Key risk: R7 and R8 are terminal today. Together they convert every ambiguous or crash-tinged failure into report_only **before** the App Profile / Repo Intelligence advisors ever get a chance to ground a selector. This is the architectural seam you're pointing at.

Stage 4 — Worker routing (`src/api/server.ts` , `createHealingWorker`)

#	Location	Exit	Evidence	Could locator healing still succeed?	Verdict
W1	<code>isCancelled() → break (L801, 1382, 1487)</code>	break	user cancel	n/a	HARD STOP ✓
W2	<code>jobBudgetExhausted() / testBudgetExhausted() → break (L805, 1386, 1487)</code>	break	wall-clock	Yes — ran out of time, not options	HARD STOP (acceptable) but should be reported as <code>timed_out</code> , not <code>not_healed</code>
W3	<code>failure-Type==='navigation' → skip (L1096)</code>	skip+restore	analyzer type	No	HARD STOP ✓
W4	<code>failure-Type==='assertion'\\ \\ 'timeout' → wait-inject or skip (L1102)</code>	skip (locator loop never entered)	analyzer type	timeout: maybe — generic timeout can hide a locator wait	GREY for <code>timeout</code>
W5	<code>strategyPlan && !shouldAttemptLocatorHealing → skip (L1186)</code>	skip+restore	router verdict (R1-R8)	inherits all router GREYs	GREY — this is where R7/R8 become a real skip
W6	Deterministic <code>no_profile → trail.skip, fall through (L1355)</code>	not terminal (falls to intelligent pipeline)	no App Profile	Yes (intelligent pipeline still runs)	CORRECT ✓
W7	<code>ranked.candidates.length</code>	break	no advisor produced a syntactically	No (truly nothing to try)	HARD STOP (but A3 star-try)

#	Location	Exit	Evidence	Could locator healing still succeed?	Verdict
	===0 → break (L1461)		valid candidate		validation can cause this)
W8	acceptCandidate pre-check reject → continue (L1541)	continue	static acceptance	tries next candidate	CORRECT ✓ (per-candidate, not pipeline)
W9	validation-Layer.validate not approved → continue (L1556)	continue	static validation	tries next candidate	CORRECT ✓
W10	Confirmation rerun crashed before tests → revert+continue (L1817)	revert	no results file, exit≠0	Yes — environment crash reverted a possibly-good fix	GREY — environment failure should not count as candidate failure
W11	Same locator still failing → revert+continue (L1930)	revert	rerun still red on same locator	tries next candidate	CORRECT ✓
W12	!locator-Fixed after retries → break (L1980)	break	exhausted candidates	No	HARD STOP ✓

Cross-cutting findings (the architectural ones)

CC-1 — “No verdict” is silently treated as “the test’s verdict”

E2/E4/A1/W10 all produce absence of a passing result. The pipeline collapses two very different facts into one:

- “the locator is still broken” (a real healing signal), and
- “the run never produced a trustworthy result” (an environment fact).

A framework crash / timeout / missing-report should yield `inconclusive`, not `framework` / `not_healed`. Today there is no `inconclusive` state.

CC-2 — `healableByLocatorSwap` is a single chokepoint, set by the weakest evidence

Every grounded advisor (Repo Intelligence, App Profile, DOM Memory) is gated behind one boolean computed in the classifier from regex + (optional) evidence. When evidence is absent (C5) the regex wins. In an advisor model this boolean should be **one advisor’s opinion**, not the gate for all of them.

CC-3 — Terminal categories run BEFORE grounded advisors

`framework` (R7) and `unknown` (R8) terminate at the router — upstream of the App Profile / Repo Intelligence resolution. So the very intelligence that could disprove the “framework/unknown” guess never runs. Order is inverted relative to the product promise (“Repo Intelligence first”).

CC-4 — First-match-wins on a JOINED multi-error string

`error_message` is `errors[].message.join('\n\n')`. `detectFailureType` and the `looksLike*` helpers scan the whole blob; whichever pattern hits first wins. A genuine `waiting for locator('#x')` followed by a teardown `Target closed` can be classified `framework` because the framework regex matched somewhere in the blob.

Recommended exit taxonomy (no code yet — for discussion)

Exit class	Examples	Desired behavior
HARD STOP	assertion, navigation, api, environment, cancel, no-package.json	terminate, report honestly (these are correct today)
INCONCLUSIVE (new)	timeout w/ no results, framework crash, missing report, rerun crashed-before-tests	do NOT label as test verdict; retry once clean OR mark inconclusive; never count as “locator unhealable”
ADVISOR (downgrade from stop)	<code>unknown</code> , <code>framework</code> -as-sole-signal, low-confidence, null locator	emit signal + score, STILL run Repo Intelligence + App Profile grounding before giving up
PER-CANDIDATE (already correct)	acceptance/validation reject, same-locator-still-failing	continue to next candidate

Bottom line

- The **hard stops for assertion/navigation/api/environment are correct.**
- The **dangerous exits are the “ambiguous → terminal” ones:** `framework` (R7), `unknown` (R8), low-confidence (R1), and every “no trustworthy result” path (E2/E4/A1/W10), plus the **null-locator starvation** (A3) and the **evidence-skipped-when-locatorState-null** behavior (C5).
- All of these terminate **before** Repo Intelligence / App Profile grounding — which is exactly the inversion an advisor model would fix: let each advisor vote, treat “no result” as inconclusive (not failure), and only declare “unhealable” after the grounded advisors have actually run.

RESOLUTION — the two architectural changes (implemented)

The audit above identified many individual premature-exit paths (R1, R7, R8, C5, A3, E2/E4, ...). Rather than patch each one, we made **two architectural changes** that make whole classes of those exits impossible by construction. Individual patches were intentionally NOT applied — they disappear naturally under the new architecture.

Change 1 — Gate → Advisor (`src/core/healing-strategy-router.ts`)

The router used to be a **gate**: a regex classification (`framework` , `unknown`) or a low-confidence score could terminate healing (`report_only`) before any grounded advisor (Repo Intelligence, App Profile, DOM Memory, Reuse, AI) ran.

Now the router is an **advisor dispatcher**:

- `HARD_STOP_CATEGORIES` is restricted to the four genuine non-healable classes: **assertion, navigation, api, environment**. Only these return `report_only` with `disposition: 'hard_stop'`.
- `framework` , `unknown` , and any locator failure without a concrete locator now return an **advisor plan** (`shouldAttemptLocatorHealing: true` , `disposition: 'advisor'`) — they flow into the grounded advisor pipeline instead of terminating. No regex can stop healing before the advisors vote.
- timing failures return `disposition: 'inject_wait'` (never a locator swap).
- The old `minActionConfidence()` floor and the low-confidence Guard were removed: confidence is now an advisor input, never a terminator.

A new `disposition: HealingDisposition` field (`'hard_stop' | 'advisor' | 'inject_wait'`) on `HealingStrategyPlan` makes the decision explicit and testable.

Change 2 — INCONCLUSIVE run state (`src/core/execution-trust.ts`)

“Absence of a passing result is NOT the same as a failing test.” A run that crashed, timed out with no evidence, lost its results file, hit a missing binary, or failed to load a spec produced **no verdict** — labelling that `framework` / `fail` / “unhealable locator” was dishonest.

- New pure module `assessRunTrust()` inspects raw run facts (exit code, results file present, failure artifacts parsed, spec-load errors) and returns a

`RunTrustAssessment` with a machine `signal` (`command_not_found` / `timeout_no_evidence` / `no_results_file` / `crashed_before_tests` / `spec_load_error`) and an `inconclusive` flag.

- New `'inconclusive'` `TestOutcome` and `ResultCounts.inconclusive` counter.
- The worker (`src/api/server.ts`) now **retries an untrustworthy run once**; if it is still untrustworthy and has no failure artifact to heal, it finalizes the run as **inconclusive** (telling the user) instead of inventing a verdict.

Preserved hard stops (unchanged, correct)

assertion, navigation, api, environment-config, user cancellation, missing dependencies — these still terminate and report honestly.

Tests

- `tests/unit/healing-strategy-router.test.ts` — advisor routing + preserved hard stops (14 tests).
- `tests/unit/execution-trust.test.ts` — every trust/inconclusive signal (10 tests).